

FULL STACK DEVELOPER
INTERVIEW LIBRARY

TypeScript Interview Handbook

Book 05
Basic to Advanced TypeScript

Contents

1. TypeScript Fundamentals
2. Primitive Types and Inference
3. Arrays, Tuples and Enums
4. Interfaces and Type Aliases
5. Functions and Overloads
6. Unions, Narrowing and Guards
7. Generics
8. Classes and Object-Oriented TypeScript
9. Advanced Types
10. Utility Types and Modules
11. Compiler Configuration and Tooling
12. Coding, Debugging and Scenario Questions

Book 05 | 120 Interview Questions | Short Answers + Examples + Tips

1. TypeScript Fundamentals

Q1. What is TypeScript?

Answer: TypeScript is a statically typed superset of JavaScript that compiles to JavaScript.

Example:

```
let message: string = 'Hello';
```

Interview Tip: Mention that valid JavaScript is generally valid TypeScript.

Q2. Why use TypeScript?

Answer: It catches many errors during development, improves editor support and makes large codebases easier to maintain.

Example:

```
const total: number = price * quantity;
```

Interview Tip: Connect the benefit to safer refactoring and team development.

Q3. Does TypeScript run directly in the browser?

Answer: No. TypeScript is transpiled to JavaScript, and the generated JavaScript runs in the browser or runtime.

Example:

```
tsc app.ts
```

Interview Tip: Separate compile-time behavior from runtime behavior.

Q4. What does the TypeScript compiler do?

Answer: It checks types and transforms TypeScript syntax into JavaScript according to compiler options.

Example:

```
tsc --project tsconfig.json
```

Interview Tip: The compiler can emit JavaScript even when errors exist unless configured otherwise.

Q5. What is tsconfig.json?

Answer: It is the project configuration file for compiler options, included files and excluded files.

Example:

```
{ "compilerOptions": { "strict": true } }
```

Interview Tip: Know common options such as target, module and strict.

Q6. What is type checking?

Answer: Type checking verifies that values and operations are compatible with declared or inferred types.

Example:

```
let age: number = 30;
```

Interview Tip: Type checking happens before runtime.

Q7. What is transpilation?

Answer: Transpilation converts source code from one language level or syntax form into another, such as TypeScript to JavaScript.

Example:

```
tsc --target ES2020
```

Interview Tip: Type checking and code transformation are related but distinct.

Q8. What is the difference between TypeScript and JavaScript?

Answer: JavaScript is the runtime language. TypeScript adds static types and development-time tooling, then compiles to JavaScript.

Example:

```
function add(a: number, b: number) { return a + b; }
```

Interview Tip: Do not say TypeScript replaces JavaScript at runtime.

Q9. What is the TypeScript playground?

Answer: It is an online editor used to test TypeScript, inspect errors and view emitted JavaScript.

Example:

```
const value: string = "TS";
```

Interview Tip: Useful for demonstrations, not a substitute for project configuration.

Q10. What is structural typing?

Answer: Compatibility is based on an object's shape rather than its declared name.

Example:

```
let p: {x:number}; p = {x:1, y:2};
```

Interview Tip: TypeScript is structurally typed, unlike nominally typed languages.

2. Primitive Types and Inference

Q11. What are the common primitive types in TypeScript?

Answer: string, number, boolean, bigint, symbol, null and undefined are common primitive types.

Example:

```
let active: boolean = true;
```

Interview Tip: Primitive types use lowercase names.

Q12. What is type inference?

Answer: TypeScript determines a type automatically from the assigned value and surrounding context.

Example:

```
let count = 0; // number
```

Interview Tip: Avoid unnecessary annotations when inference is clear.

Q13. What is contextual typing?

Answer: The surrounding code supplies the expected type, such as the parameter type of a callback.

Example:

```
[1,2].map(n => n * 2);
```

Interview Tip: Common in callbacks and event handlers.

Q14. What is a literal type?

Answer: A literal type represents one exact value rather than the wider primitive type.

Example:

```
let direction: 'left' | 'right';
```

Interview Tip: Literal types are useful for controlled options.

Q15. What is type widening?

Answer: TypeScript may widen a literal value to a broader type when the value can change.

Example:

```
let status = 'ready'; // string
```

Interview Tip: const often preserves narrower literal information.

Q16. What is const assertion?

Answer: as const makes literals readonly and preserves their narrow literal types.

Example:

```
const roles = ['admin', 'user'] as const;
```

Interview Tip: It is a compile-time assertion, not runtime freezing.

Q17. What is the any type?

Answer: any disables most type checking for that value and allows arbitrary operations.

Example:

```
let data: any = response;
```

Interview Tip: Use sparingly because it removes safety.

Q18. What is the unknown type?

Answer: unknown can hold any value, but it must be narrowed before use.

Example:

```
let value: unknown = input;
```

Interview Tip: Prefer unknown over any for untrusted values.

Q19. What is the never type?

Answer: never represents values that cannot occur, such as a function that always throws or an impossible branch.

Example:

```
function fail(msg:string): never { throw new Error(msg); }
```

Interview Tip: Useful for exhaustive checks.

Q20. What is the void type?

Answer: void describes a function whose return value is not intended to be used.

Example:

```
function log(msg:string): void { console.log(msg); }
```

Interview Tip: A JavaScript function may still return undefined.

3. Arrays, Tuples and Enums

Q21. How do you type an array?

Answer: Use Type[] or Array<Type>.

Example:

```
const ids: number[] = [1,2,3];
```

Interview Tip: Both syntaxes are equivalent for ordinary arrays.

Q22. What is a tuple?

Answer: A tuple is an array type with a fixed sequence of element types.

Example:

```
const user: [number, string] = [1, "Sam"];
```

Interview Tip: Tuple length and position matter.

Q23. Can tuples have optional elements?

Answer: Yes. Optional tuple elements use a question mark and must come after required elements.

Example:

```
type Pair = [number, string?];
```

Interview Tip: Optional positions affect possible tuple lengths.

Q24. What is a readonly array?

Answer: A readonly array prevents mutation through that reference.

Example:

```
const items: readonly string[] = ["a", "b"];
```

Interview Tip: It does not deeply freeze nested objects.

Q25. What is a readonly tuple?

Answer: A readonly tuple preserves tuple positions while preventing mutation.

Example:

```
const point: readonly [number, number] = [10, 20];
```

Interview Tip: as const commonly creates readonly tuples.

Q26. What is an enum?

Answer: An enum defines a named set of related constants.

Example:

```
enum Status { Pending, Done }
```

Interview Tip: Consider union literals for lightweight alternatives.

Q27. Numeric enum vs string enum?

Answer: Numeric enums use numbers and auto-increment; string enums use explicit string values.

Example:

```
enum Role { Admin='ADMIN', User='USER' }
```

Interview Tip: String enums are often clearer in logs and APIs.

Q28. What is a const enum?

Answer: A const enum is inlined by the compiler to reduce emitted enum code.

Example:

```
const enum Direction { Left, Right }
```

Interview Tip: It can create issues across package boundaries and some build tools.

Q29. How do you get a union from a readonly array?

Answer: Use indexed access with typeof and number.

Example:

```
const sizes=['S','M','L'] as const; type Size = typeof sizes[number];
```

Interview Tip: A common pattern for runtime values plus compile-time types.

Q30. Array vs tuple?

Answer: An array has a repeated element type and flexible length; a tuple models a fixed positional structure.

Example:

```
type RGB = [number, number, number];
```

Interview Tip: Use objects when named fields are clearer than positions.

4. Interfaces and Type Aliases

Q31. What is an interface?

Answer: An interface describes the shape of an object, class or callable structure.

Example:

```
interface User { id:number; name:string; }
```

Interview Tip: Interfaces are commonly used for public object contracts.

Q32. What is a type alias?

Answer: A type alias gives a name to any type expression, including unions, tuples and primitives.

Example:

```
type ID = string | number;
```

Interview Tip: Type aliases are more flexible for non-object types.

Q33. Interface vs type alias?

Answer: Both describe shapes. Interfaces support declaration merging; type aliases naturally express unions, tuples and mapped types.

Example:

```
type Result = Success | Failure;
```

Interview Tip: Choose consistency and the feature needed.

Q34. What is declaration merging?

Answer: Multiple interface declarations with the same name are combined.

Example:

```
interface Window { appVersion: string; }
```

Interview Tip: This is useful for augmenting library or global types.

Q35. How do interfaces extend other interfaces?

Answer: Use extends to inherit members from one or more interfaces.

Example:

```
interface Admin extends User { permissions:string[]; }
```

Interview Tip: Extension keeps shared contracts reusable.

Q36. What is an intersection type?

Answer: An intersection combines multiple types into one type containing all required members.

Example:

```
type Employee = Person & Worker;
```

Interview Tip: Conflicting properties may produce never.

Q37. What is an optional property?

Answer: A property followed by ? may be absent.

Example:

```
interface Config { timeout?: number; }
```

Interview Tip: With strict settings, absent and undefined may need careful handling.

Q38. What is a readonly property?

Answer: A readonly property cannot be reassigned through that type after initialization.

Example:

```
interface Point { readonly x:number; }
```

Interview Tip: Readonly is compile-time only.

Q39. What is an index signature?

Answer: An index signature describes properties whose names are not known in advance.

Example:

```
interface Scores { [name:string]: number; }
```

Interview Tip: Known properties must be compatible with the index value type.

Q40. What are excess property checks?

Answer: Object literals are checked for unexpected properties when assigned to a target object type.

Example:

```
const u: User = { id:1, name:"A" };
```

Interview Tip: Variables and assertions can behave differently from direct literals.

5. Functions and Overloads

Q41. How do you type function parameters?

Answer: Add type annotations after parameter names.

Example:

```
function greet(name: string) { return `Hi ${name}`; }
```

Interview Tip: Prefer inference for obvious return types.

Q42. How do you type a function return value?

Answer: Add the return type after the parameter list.

Example:

```
function square(n:number): number { return n*n; }
```

Interview Tip: Explicit returns help public APIs.

Q43. What is a function type expression?

Answer: It describes a callable value using parameter and return types.

Example:

```
type Handler = (event: Event) => void;
```

Interview Tip: Parameter names improve readability but do not affect compatibility.

Q44. What are optional parameters?

Answer: A parameter marked ? may be omitted by callers.

Example:

```
function log(message:string, level?:string) {}
```

Interview Tip: Required parameters cannot follow optional ones.

Q45. What are default parameters?

Answer: A default value is used when the argument is omitted or undefined.

Example:

```
function greet(name = "Guest") {}
```

Interview Tip: The parameter is treated as optional to callers.

Q46. What are rest parameters?

Answer: Rest parameters collect remaining arguments into an array.

Example:

```
function sum(...values:number[]) { return values.reduce((a,b)=>a+b,0); }
```

Interview Tip: The rest parameter must be last.

Q47. What are function overloads?

Answer: Overloads provide multiple call signatures for one implementation.

Example:

```
function parse(x:string):number; function parse(x:number):string;
```

Interview Tip: The implementation signature is not directly callable.

Q48. When should overloads be used?

Answer: Use overloads when different inputs produce meaningfully different output types.

Example:

```
function create(value:string):Text; function create(value:number):NumberBox;
```

Interview Tip: Prefer a union parameter when the return type is the same.

Q49. What is a call signature in an object type?

Answer: It describes a callable object that may also have properties.

Example:

```
type Fn = { (x:number):number; label:string };
```

Interview Tip: Useful for functions with metadata.

Q50. What is a construct signature?

Answer: It describes a value that can be called with new.

Example:

```
type Ctor = new (name:string) => User;
```

Interview Tip: Useful for factories that accept classes.

6. Unions, Narrowing and Guards

Q51. What is a union type?

Answer: A union allows a value to be one of several types.

Example:

```
let id: string | number;
```

Interview Tip: Only operations valid for every union member are allowed before narrowing.

Q52. What is type narrowing?

Answer: Narrowing reduces a broad type to a more specific type through control-flow analysis.

Example:

```
if (typeof id === "string") id.toUpperCase();
```

Interview Tip: TypeScript follows branches and assignments.

Q53. How does typeof narrow types?

Answer: typeof checks narrow primitive-like unions such as string, number and boolean.

Example:

```
if (typeof value === "number") value.toFixed(2);
```

Interview Tip: Remember typeof null is object at runtime.

Q54. How does instanceof narrow types?

Answer: instanceof narrows values based on constructor relationships.

Example:

```
if (error instanceof Error) console.log(error.message);
```

Interview Tip: Useful for classes and built-in objects.

Q55. How does the in operator narrow types?

Answer: The in operator checks whether a property exists and narrows compatible union members.

Example:

```
if ("swim" in pet) pet.swim();
```

Interview Tip: Optional properties may appear on both sides.

Q56. What is a custom type predicate?

Answer: A function return type of parameter is Type tells the compiler that a successful result narrows the parameter.

Example:

```
function isUser(x:unknown): x is User { return typeof x === "object" && x !== null && "id" in x; }
```

Interview Tip: The runtime logic must genuinely validate the claim.

Q57. What is a discriminated union?

Answer: It is a union whose members share a literal property used to identify each variant.

Example:

```
type Shape = {kind:"circle"; r:number} | {kind:"square"; s:number};
```

Interview Tip: Excellent for state machines and API results.

Q58. What is an exhaustive check?

Answer: An exhaustive check ensures every union variant is handled, often by assigning the remaining value to `never`.

Example:

```
const _never: never = shape;
```

Interview Tip: It catches missing cases when unions grow.

Q59. What is a truthiness narrowing?

Answer: Conditions such as `if(value)` remove falsy possibilities from the current branch.

Example:

```
if (name) console.log(name.length);
```

Interview Tip: Be careful because empty strings and zero are falsy.

Q60. What is equality narrowing?

Answer: Comparing values can narrow them to compatible overlapping types.

Example:

```
if (a === b) { /* shared type */ }
```

Interview Tip: Strict equality gives clearer runtime behavior.

7. Generics

Q61. What is a generic?

Answer: A generic uses type parameters so code can work with multiple types while preserving type information.

Example:

```
function identity<T>(value:T): T { return value; }
```

Interview Tip: Generics connect input and output types.

Q62. Why use generics instead of `any`?

Answer: Generics retain relationships and safety, while `any` discards type information.

Example:

```
const value = identity<string>("hello");
```

Interview Tip: Let inference choose type arguments when possible.

Q63. What is a generic constraint?

Answer: A constraint limits which types may be supplied to a type parameter.

Example:

```
function lengthOf<T extends {length:number}>(x:T) { return x.length; }
```

Interview Tip: Constraints allow safe member access.

Q64. What does `keyof` do with generics?

Answer: `keyof` creates a union of known property keys and can constrain key parameters.

Example:

```
function get<T, K extends keyof T>(obj:T, key:K): T[K] { return obj[key]; }
```

Interview Tip: This pattern preserves the selected property type.

Q65. What are generic interfaces?

Answer: Interfaces can accept type parameters for reusable contracts.

Example:

```
interface ApiResponse<T> { data:T; error?:string; }
```

Interview Tip: Use meaningful parameter names in complex APIs.

Q66. What are generic classes?

Answer: Classes may use type parameters for their fields and methods.

Example:

```
class Box<T> { constructor(public value:T) {} }
```

Interview Tip: Static members cannot use the class type parameter directly.

Q67. What are default generic types?

Answer: A type parameter can have a default used when no type argument is supplied.

Example:

```
type Api<T = unknown> = { data:T };
```

Interview Tip: Defaults should be safe and unsurprising.

Q68. Can generics have multiple type parameters?

Answer: Yes. Multiple parameters model relationships among several types.

Example:

```
function pair<K,V>(key:K, value:V): [K,V] { return [key,value]; }
```

Interview Tip: Keep the number of parameters manageable.

Q69. What is generic inference?

Answer: The compiler infers type arguments from function arguments and context.

Example:

```
const result = identity(42); // number
```

Interview Tip: Explicit arguments are needed only when inference is insufficient.

Q70. What is variance in TypeScript?

Answer: Variance describes how generic type compatibility changes when the type argument changes.

Example:

```
type Producer<T> = () => T;
```

Interview Tip: Function parameter variance depends on compiler settings and method syntax.

8. Classes and Object-Oriented TypeScript

Q71. How do you declare a class?

Answer: Use class with fields, a constructor and methods.

Example:

```
class User { constructor(public name:string) {} }
```

Interview Tip: Classes exist at runtime and also create instance types.

Q72. What are access modifiers?

Answer: public, protected and private control member visibility in TypeScript.

Example:

```
class A { private secret = 1; }
```

Interview Tip: TypeScript private is usually compile-time only.

Q73. What are ECMAScript private fields?

Answer: Fields beginning with # are enforced as private by JavaScript at runtime.

Example:

```
class A { #secret = 1; }
```

Interview Tip: They differ from the private keyword.

Q74. What are parameter properties?

Answer: A constructor parameter with an access modifier or readonly automatically becomes a class property.

Example:

```
constructor(public id:number, private token:string) {}
```

Interview Tip: They reduce boilerplate.

Q75. What is an abstract class?

Answer: An abstract class cannot be instantiated directly and may declare abstract members for subclasses.

Example:

```
abstract class Shape { abstract area():number; }
```

Interview Tip: Use it when shared implementation and a base contract are both needed.

Q76. What is an abstract method?

Answer: An abstract method declares a required method signature without implementation in the base class.

Example:

```
abstract render(): void;
```

Interview Tip: Subclasses must implement it unless they are also abstract.

Q77. How does a class implement an interface?

Answer: Use implements to check that the class instance satisfies the interface.

Example:

```
class Admin implements UserContract { id=1; }
```

Interview Tip: implements does not copy implementation.

Q78. What is method overriding?

Answer: A subclass provides a method with the same contract as a base-class method.

Example:

```
class Dog extends Animal { override speak() {} }
```

Interview Tip: The override keyword can catch accidental mismatches.

Q79. What is a static member?

Answer: A static member belongs to the class constructor rather than each instance.

Example:

```
class MathUtil { static double(n:number){return n*2;} }
```

Interview Tip: Access it through the class name.

Q80. What is a protected member?

Answer: A protected member is accessible inside the declaring class and subclasses, but not from ordinary outside code.

Example:

```
class Base { protected value=1; }
```

Interview Tip: It supports inheritance while hiding public access.

9. Advanced Types

Q81. What is keyof?

Answer: keyof produces a union of property names from an object type.

Example:

```
type UserKey = keyof User;
```

Interview Tip: Numeric and symbol keys may also appear.

Q82. What is typeof in a type position?

Answer: It captures the compile-time type of a value.

Example:

```
const config={mode:"dark"}; type Config=typeof config;
```

Interview Tip: It does not run the JavaScript typeof operator.

Q83. What is indexed access type?

Answer: It retrieves a property type from another type using bracket notation.

Example:

```
type Name = User["name"];
```

Interview Tip: A union of keys produces a union of property types.

Q84. What is a mapped type?

Answer: A mapped type creates a new type by iterating over a union of property keys.

Example:

```
type ReadonlyUser = { readonly [K in keyof User]: User[K] };
```

Interview Tip: Utility types are built from this idea.

Q85. What is a conditional type?

Answer: A conditional type chooses one type or another based on assignability.

Example:

```
type IsString<T> = T extends string ? true : false;
```

Interview Tip: Conditional types distribute over naked union type parameters.

Q86. What is infer in a conditional type?

Answer: infer introduces a type variable extracted from a matched structure.

Example:

```
type Return<T> = T extends (...a:any[])=>infer R ? R : never;
```

Interview Tip: Use it inside the true branch of a conditional type.

Q87. What are template literal types?

Answer: They build string literal types by combining literal unions.

Example:

```
type EventName = `on${Capitalize<string>}`;
```

Interview Tip: Useful for strongly typed naming conventions.

Q88. What is satisfies?

Answer: satisfies checks that an expression conforms to a type while preserving its more specific inferred type.

Example:

```
const routes = {home:"/"} satisfies Record<string,string>;
```

Interview Tip: It differs from a type annotation or assertion.

Q89. What is a branded type pattern?

Answer: A branded type adds a unique marker to distinguish values that share the same runtime representation.

Example:

```
type UserId = string & {readonly __brand:"UserId"};
```

Interview Tip: It simulates nominal typing at compile time.

Q90. What is recursive type?

Answer: A recursive type refers to itself to model nested structures.

Example:

```
type Tree<T> = { value:T; children:Tree<T>[] };
```

Interview Tip: Guard against excessively deep type instantiation.

10. Utility Types and Modules

Q91. What does Partial<T> do?

Answer: It makes every property in T optional.

Example:

```
type UserPatch = Partial<User>;
```

Interview Tip: Useful for update payloads, but validate required business rules.

Q92. What does Required<T> do?

Answer: It makes every property in T required.

Example:

```
type Complete = Required<Config>;
```

Interview Tip: It removes optional modifiers only at the top level.

Q93. What does Readonly<T> do?

Answer: It makes every property in T readonly.

Example:

```
type Locked = Readonly<User>;
```

Interview Tip: It is shallow.

Q94. What does Pick<T, K> do?

Answer: It creates a type containing only selected keys from T.

Example:

```
type UserPreview = Pick<User, "id" | "name">;
```

Interview Tip: Useful for view models.

Q95. What does Omit<T, K> do?

Answer: It creates a type by removing selected keys from T.

Example:

```
type NewUser = Omit<User, "id">;
```

Interview Tip: Prefer explicit API types when semantics differ greatly.

Q96. What does Record<K, V> do?

Answer: It creates an object type whose keys are K and values are V.

Example:

```
type Scores = Record<string, number>;
```

Interview Tip: Literal key unions can require every key.

Q97. What do Exclude and Extract do?

Answer: Exclude removes union members assignable to another type; Extract keeps matching members.

Example:

```
type A = Exclude<"a" | "b", "a">;
```

Interview Tip: They operate on unions.

Q98. What does NonNullable<T> do?

Answer: It removes null and undefined from T.

Example:

```
type Name = NonNullable<string|null>;
```

Interview Tip: It does not perform a runtime check.

Q99. What are ES modules in TypeScript?

Answer: Files with import or export have module scope and can share typed values across files.

Example:

```
export interface User {}; import type {User} from "../user";
```

Interview Tip: Use import type when only the type is needed.

Q100. What is a declaration file?

Answer: A .d.ts file describes types for JavaScript code without emitting runtime JavaScript.

Example:

```
declare module "legacy-lib";
```

Interview Tip: DefinitelyTyped packages commonly provide @types definitions.

11. Compiler Configuration and Tooling

Q101. What does strict mode in tsconfig enable?

Answer: strict enables a family of stronger type-checking options.

Example:

```
{ "compilerOptions": { "strict": true } }
```

Interview Tip: It is a strong default for new projects.

Q102. What is strictNullChecks?

Answer: It treats null and undefined as distinct types that must be handled explicitly.

Example:

```
let name: string | null = null;
```

Interview Tip: This prevents many common runtime errors.

Q103. What is noImplicitAny?

Answer: It reports places where the compiler would otherwise infer any implicitly.

Example:

```
{ "noImplicitAny": true }
```

Interview Tip: It encourages explicit safe types.

Q104. What does target control?

Answer: target chooses the JavaScript language version emitted by the compiler.

Example:

```
{ "target": "ES2020" }
```

Interview Tip: Runtime support and polyfills are separate concerns.

Q105. What does module control?

Answer: module chooses the generated module format or module behavior.

Example:

```
{ "module": "NodeNext" }
```

Interview Tip: Match it to the runtime and bundler.

Q106. What is moduleResolution?

Answer: It controls how import specifiers are resolved to files and packages.

Example:

```
{ "moduleResolution": "Bundler" }
```

Interview Tip: Use a strategy compatible with your environment.

Q107. What does esModuleInterop do?

Answer: It improves compatibility for importing many CommonJS modules with ES-style syntax.

Example:

```
{ "esModuleInterop": true }
```

Interview Tip: Understand the emitted interop helpers.

Q108. What does noEmit do?

Answer: It performs type checking without generating JavaScript output.

Example:

```
{ "noEmit": true }
```

Interview Tip: Common when a bundler or framework handles transpilation.

Q109. What are source maps?

Answer: Source maps connect generated JavaScript back to TypeScript for debugging.

Example:

```
{ "sourceMap": true }
```

Interview Tip: Do not expose sensitive source maps unintentionally in production.

Q110. What is incremental compilation?

Answer: It stores build information so later compilations can reuse previous work.

Example:

```
{ "incremental": true }
```

Interview Tip: Useful for large projects and project references.

12. Coding, Debugging and Scenario Questions

Q111. How would you safely parse unknown API data?

Answer: Keep the value unknown, validate its runtime shape, then narrow it to a trusted type.

Example:

```
const data: unknown = await response.json(); if (isUser(data)) useUser(data);
```

Interview Tip: Static types do not validate network data at runtime.

Q112. How do you fix Object is possibly undefined?

Answer: Narrow the value, use optional chaining, provide a default or redesign the type so absence is explicit.

Example:

```
const city = user.address?.city ?? "Unknown";
```

Interview Tip: Avoid non-null assertions unless the invariant is guaranteed.

Q113. What is a non-null assertion?

Answer: The postfix ! tells TypeScript to treat a value as non-nullish without a runtime check.

Example:

```
element!.focus();
```

Interview Tip: Overuse can hide real bugs.

Q114. What is a type assertion?

Answer: A type assertion tells the compiler to treat a value as another compatible type.

Example:

```
const input = document.querySelector("input") as HTMLInputElement;
```

Interview Tip: Assertions do not convert or validate values.

Q115. Why can a type assertion be dangerous?

Answer: It can silence errors even when the runtime value does not match the asserted type.

Example:

```
const user = data as User;
```

Interview Tip: Validate external data instead of blindly asserting.

Q116. How do you type an event handler in the browser?

Answer: Use the appropriate DOM event and element types, or rely on contextual typing.

Example:

```
input.addEventListener("input", (e: Event) => { const el=e.currentTarget as HTMLInputElement;
});
```

Interview Tip: currentTarget is often safer than target for the listener element.

Q117. How do you model loading, success and error states?

Answer: Use a discriminated union with one variant for each state.

Example:

```
type State<T>={status:"loading"}|{status:"success";data:T}|{status:"error";message:string};
```

Interview Tip: This prevents impossible combinations of fields.

Q118. How do you create a type-safe property getter?

Answer: Use generics with K extends keyof T and return T[K].

Example:

```
function get<T,K extends keyof T>(o:T,k:K):T[K]{return o[k];}
```

Interview Tip: This is a frequent generic interview question.

Q119. How do you make selected properties optional?

Answer: Combine Omit with Partial<Pick<...>>.

Example:

```
type Flexible<T,K extends keyof T> = Omit<T,K> & Partial<Pick<T,K>>;
```

Interview Tip: Mapped utility combinations are common in reusable libraries.

Q120. How do you prevent an unhandled union case?

Answer: Use a switch on the discriminant and a never assertion in the default branch.

Example:

```
default: return assertNever(value);
```

Interview Tip: This converts future union changes into compile-time feedback.